INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HYDERABAD

AIML B26 - PGCP

AI-Powered Retail Shelf Intelligence System

Multi-Stage Computer Vision Pipeline with
Planogram Compliance & LLM Fallback

Moorthy · Nikhil · Sivakanth · Surabi

Project Mentor: Pratyusha Mitra

Project Supervisor: Professor Anoop Namboodiri

August 2026

Table of Contents

1. INTRODUCTION

Retail inventory management is one of the most operationally intensive activities in modern commerce. Retailers lose billions of dollars annually due to out-of-stock events, misplaced products, and non-compliance with planogram specifications. Traditional manual counting is slow, error-prone, and fails to scale with the pace of modern retail.

This project proposes and demonstrates an end-to-end AI-powered retail shelf intelligence system that automates product detection, identification, and planogram compliance verification using a multi-stage computer vision pipeline. The system combines YOLOv8n for object detection, CLIP (Contrastive Language-Image Pre-training) for both category classification and fine-grained SKU matching, a planogram-based grid lookup for shelf-position identification, and a cascading LLM fallback (GPT-4o → Gemini 2.0 Flash → Groq LLaMA-3) for products that remain unidentified after all prior stages. The system is deployed as a full-stack web application on Hugging Face Spaces using Docker.

1.1 Problem Statement

Traditional retail inventory management faces several critical challenges:

- Manual product counting is time-consuming, error-prone, and requires significant labor hours
- Real-time inventory tracking at the SKU level is difficult to maintain without dedicated hardware
- Planogram compliance — verifying that products are placed in their designated shelf positions — requires trained staff who physically walk every aisle
- Identifying new or unknown products on shelves requires manual lookup and cannot be automated without expensive barcode infrastructure
- Business intelligence queries over inventory data require SQL expertise, limiting access for store managers

1.2 Project Objectives

This project aims to address the above challenges by developing an AI-powered system with the following objectives:

- 1. Automatically detect all products on a retail shelf image using YOLOv8n
- 2. Classify each detected product crop into a retail category using zero-shot CLIP text-similarity prompts (38 categories)
- 3. Perform fine-grained SKU identification by matching CLIP embeddings against a growing product library
- 4. Verify planogram compliance by mapping detected products to their expected grid positions using a planogram database
- 5. Fall back to a cascade of LLMs (GPT-4o, Gemini 2.0 Flash, Groq LLaMA-3) for products unidentified after library and planogram lookup
- 6. Provide a user-friendly web interface for image upload, result visualization, and product library management
- 7. Deploy the complete system publicly on Hugging Face Spaces for demonstration and evaluation

1.3 Scope

In Scope:

- Object detection and product identification for retail shelf images
- 38-class zero-shot category classification using CLIP text prompts
- Fine-grained SKU matching using 512-dimensional CLIP embeddings with cosine similarity
- Planogram compliance verification using a shelf grid database
- Multi-provider LLM fallback for unknown products
- Full-stack web application with JWT authentication and Docker deployment

Out of Scope: Physical hardware integration (barcode scanners, RFID, IP cameras), real-time video stream processing, payment or POS system integration, multi-store synchronization.

1.4 Expected Outcomes

- ≥ 90% product detection coverage on standard retail shelf images
- ≥ 85% SKU identification rate (combined library + planogram + LLM)
- 100% planogram compliance check for shelves with a loaded planogram
- End-to-end pipeline latency under 10 seconds for a typical shelf image
- A publicly accessible demo deployed on Hugging Face Spaces

2. LITERATURE REVIEW

2.1 Object Detection in Retail

Object detection has been extensively studied for retail applications. The YOLO family of detectors (Redmon et al., 2016) introduced the paradigm of single-pass, anchor-based detection that enabled real-time inference. Subsequent versions — YOLOv5 (2020), YOLOv8 (2023) — progressively improved accuracy and deployment ease. For our application, we selected YOLOv8n (nano variant) for its balance of speed and accuracy, its anchor-free architecture, and its ease of integration via the Ultralytics library.

The SKU-110K dataset (Goldman et al., 2019) demonstrated that densely packed retail shelf images pose unique challenges that standard COCO-trained models struggle with. Alternate approaches such as Faster R-CNN (Ren et al., 2015) achieve higher accuracy at the cost of significantly slower inference (100–140 ms vs. 20–30 ms for YOLO), making them unsuitable for interactive web applications.

2.2 Vision-Language Models and CLIP

The introduction of CLIP (Radford et al., 2021) by OpenAI marked a paradigm shift in image recognition. By training a vision encoder (ViT-B/32) and a text encoder jointly on 400 million image-text pairs using a contrastive objective, CLIP learns a shared embedding space where semantically related images and text are close together. This enables zero-shot classification — given a set of text prompts, CLIP can classify an image without any task-specific fine-tuning.

For our system, CLIP serves two distinct roles: (1) zero-shot category classification in Stage 2 using 38 retail category text prompts, and (2) fine-grained SKU matching in Stage 3 by comparing 512-dimensional image embeddings against a pre-built library of reference product embeddings using cosine similarity. The threshold of 0.82 was empirically selected to balance precision and recall on our demo dataset.

2.3 Planogram Compliance Systems

Planogram compliance — verifying that products are placed according to a store's intended shelf layout — has traditionally required human auditors. Our approach takes a lightweight grid-based method: the shelf image is divided into a proportional grid based on the planogram's declared row and column counts, each detected product is assigned a (row, col) position based on its bounding box center, and this position is looked up in the planogram database to retrieve the expected product name. This avoids the need for per-product trained classifiers while still providing meaningful compliance signals.

2.4 Large Language Models for Product Identification

Large language models with vision capabilities have matured rapidly. GPT-4o (OpenAI, 2024), Gemini 2.0 Flash (Google, 2024), and open-weight models like LLaMA-3 (Meta, 2024) can analyze product images and extract structured information such as brand, product name, and category. For our system, LLMs serve as a last-resort fallback when neither the CLIP library nor the planogram database can identify a product. We employ a cascading strategy across three providers to maximize uptime and minimize cost.

2.5 Deployment and MLOps

Hugging Face Spaces provides a zero-cost inference hosting platform that supports Docker-based deployments with persistent storage. FastAPI has emerged as the dominant Python web framework for ML APIs due to its automatic OpenAPI documentation, native async support, and Pydantic-based validation. React remains the most widely used framework for interactive single-page applications, making it ideal for our detection visualization interface.

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The system follows a layered architecture comprising four tiers: Client, Application, AI/ML, and Data.

Layer	Component	Technology
Client	Web SPA	React 18 + Vite + Axios
Application	REST API	FastAPI 0.110 + Pydantic v2
Application	Authentication	JWT (python-jose)
AI/ML	Object Detection	YOLOv8n (Ultralytics)
AI/ML	Category Classification	CLIP ViT-B/32 (OpenAI)
AI/ML	SKU Matching	CLIP Embeddings + Cosine Similarity
AI/ML	Planogram Lookup	SQLite Grid DB
AI/ML	LLM Fallback	GPT-4o / Gemini 2.0 Flash / Groq LLaMA-3
Data	Database	SQLite + SQLAlchemy ORM
Deployment	Hosting	Hugging Face Spaces (Docker)

3.2 Four-Stage Detection Pipeline

The core of the system is a four-stage hierarchical detection pipeline. Each stage progressively narrows the identification task, with the next stage invoked only if the previous stage fails to produce a confident identification:

Stage	Name	Model	Purpose	Threshold
1	Object Detection	YOLOv8n	Locate all products; produce bounding boxes	Conf ≥ 0.25
2	Category Classification	CLIP ViT-B/32	Zero-shot classify crop into 1 of 38 categories	Top-1 similarity
3	SKU Matching	CLIP ViT-B/32	Match 512-dim embedding against product library	Cosine ≥ 0.82
3.5	Planogram Lookup	Grid DB	Map detected position (row, col) to expected product	Exact grid match
3b	LLM Fallback	GPT-4o / Gemini / Groq	Identify unmatched crops via vision LLM	Confidence ≥ 50

3.3 Data Flow Architecture

The following describes the complete data flow for a single shelf image:

- **Step 1:** User uploads a shelf image via the Detection page
- **Step 2:** FastAPI /detection/upload saves the image and runs YOLOv8n inference, returning bounding boxes with confidence scores

- **Step 3:** The frontend sends the detection list to /detection/pipeline along with the selected shelf_id and use_planogram flag
- **Step 4:** Stage 3.5 planogram lookup runs FIRST — each detection's grid position (row, col) is computed and looked up in the planogram DB
- **Step 5:** For positions NOT covered by planogram, Stage 3 CLIP SKU matching runs (cosine similarity against library references)
- **Step 6:** For detections still unidentified, Stage 3b LLM cascade is invoked
- **Step 7:** Compliance is computed, enriched results returned, and inventory auto-updated
- **Step 8:** User can save planogram-matched items to the product library for improved future matching

4. MODEL SELECTION AND JUSTIFICATION

4.1 Object Detection: YOLOv8n

We evaluated several object detection architectures for Stage 1:

Model	mAP50	CPU Latency	Model Size	Deployment	Selected
YOLOv8n	37.3%	~30ms	6.2 MB	Easy (Ultralytics)	✓
YOLOv8s	44.9%	~55ms	21.5 MB	Easy (Ultralytics)	
Faster R-CNN	37.9%	~130ms	137 MB	Complex	
EfficientDet-D0	33.8%	~67ms	15.9 MB	Moderate	
RT-DETR-L	53.0%	~75ms	113 MB	Moderate	

YOLOv8n was selected as the optimal choice for our deployment context: the nano variant fits in 6.2 MB, runs in ~30 ms on CPU, and integrates seamlessly with the Ultralytics API. While larger models achieve higher mAP, the nano variant is sufficient for detecting product bounding boxes — the actual SKU identification work is delegated to CLIP and the LLM fallback. Configuration used: confidence threshold 0.25, IOU threshold 0.45 for NMS.

4.2 Vision-Language Model: CLIP ViT-B/32

CLIP serves two purposes: zero-shot category classification (Stage 2) and 512-dimensional embedding extraction for SKU library matching (Stage 3).

Criterion	CLIP ViT-B/32	ResNet-50	EfficientNet-B0	Justification
Semantic Understanding	Excellent	Moderate	Moderate	Cross-modal alignment
Zero-Shot Capability	Yes	No	No	No retraining needed
Embedding Dimension	512	2048	1280	Balanced size
Pre-training Scale	400M pairs	ImageNet	ImageNet	Largest dataset
Inference Speed (CPU)	20–30ms	15–20ms	25–35ms	Fast enough

4.3 Planogram Grid Lookup

The planogram system uses a lightweight approach that avoids per-product classifiers entirely. Each shelf is described by a CSV planogram file imported into SQLite. When a product is not found in the CLIP library, its bounding box center (xc, yc) is mapped to a grid position using proportional mapping:

```
col = min(int(xc / image_width × num_cols), num_cols – 1) + 1
row = min(int(yc / image_height × num_rows), num_rows – 1) + 1
```

The grid dimensions (num_rows, num_cols) are read automatically from the planogram database, so no hardcoded values are needed. Crucially, the planogram lookup now runs **before** the CLIP library search to prevent false-positive library matches across different shelf types (e.g., a beverage library matching dairy shelf products).

4.4 LLM Fallback Cascade

Products that remain unidentified after library and planogram lookup are sent to a cascading LLM pipeline:

- **GPT-4o (OpenAI)** — Primary provider: Highest accuracy, handles partial labels and varied packaging
- **Gemini 2.0 Flash (Google)** — Secondary: Fast and cost-effective, strong visual understanding
- **Groq LLaMA-3 Vision** — Tertiary: Open-weight model on Groq's low-latency inference infrastructure
- **Rule-based fallback** — Last resort: CLIP category label used as product name with low confidence flag

5. IMPLEMENTATION DETAILS

5.1 Backend Architecture (FastAPI)

The backend is implemented as a FastAPI application organized into routers: /auth (JWT login/register), /detection (upload, pipeline, library save), /library (reference image management, CLIP embedding storage), /inventory (product catalog, stock alerts), /planogram (CRUD for shelf planograms), /analytics (dashboard metrics), and /nlq (natural language query to SQL). SQLAlchemy ORM with SQLite is used for persistence. CLIP models are loaded lazily at first use and cached in memory for the process lifetime to avoid re-initialization overhead.

5.2 Detection Pipeline Implementation

The core pipeline is implemented in backend/routers/detection.py. The PipelineRequest model accepts event_id, detections list, shelf_id, and use_planogram flag.

A key implementation detail is the _assign_grid function, which uses proportional mapping to correctly assign (row, col) to each detected product:

```
def _assign_grid(dets, img_w, img_h, num_rows, num_cols):
band_h = img_h / num_rows
band_w = img_w / num_cols
for d in dets:
xc = (d['bbox'][0] + d['bbox'][2]) / 2
yc = (d['bbox'][1] + d['bbox'][3]) / 2
row = min(int(yc / band_h), num_rows - 1) + 1
col = min(int(xc / band_w), num_cols - 1) + 1
```

5.3 Planogram Compliance Matching

Compliance is determined by word-level substring matching with hyphen and underscore normalization:

```
det_low = final_name.lower().replace('-',' ').replace('_',' ')
exp_low = pog_entry.product_name.lower().replace('-',' ').replace('_',' ')
words_ok = any(w in det_low for w in exp_low.split() if len(w) > 3)
compliance = 'ok' if words_ok else 'mismatch'
```

5.4 Frontend Architecture (React)

The frontend is a React 18 SPA built with Vite and served by FastAPI's catch-all route. All API calls use Axios with a relative base URL (baseURL: ''), which ensures the app works correctly whether served from localhost or from Hugging Face Spaces.

Key frontend pages: **Detection** (image upload, pipeline trigger, results table with stage-colored rows, compliance badges, Save to Library), **Product Library** (reference image upload, CLIP embedding status), **Inventory** (product catalog with stock levels and alerts), **Analytics** (detection event history), **Planogram** (shelf planogram viewer and CSV import).

5.5 Product Library and CLIP Embedding Storage

The product library stores reference crops for each known product. When a crop is saved (via the Product Library page or the 'Save to Library' button on a planogram-matched detection), the system: extracts the bounding box crop, runs CLIP ViT-B/32 to generate a 512-dimensional L2-normalized embedding, and stores the embedding (as a binary blob) alongside the product name in the library_references table.

This 'progressive library building' workflow means the system improves over time — planogram-identified products are saved to the library, and future scans match them via fast CLIP cosine similarity. The Product Library page shows a coverage indicator per product: Good (≥3 references), Moderate (2), or Needs more refs (1).

5.6 Deployment

The complete system is containerized in a single Docker image using a Python 3.11 slim base. The Hugging Face Spaces platform automatically builds and deploys the Docker image on every git push to the main branch. A GitHub Actions workflow handles CI and automatic deployment to HF Spaces.

6. EVALUATION METRICS AND RESULTS

6.1 Object Detection Performance (YOLOv8n)

Metric	Value	Description
mAP50	37.3%	Mean Average Precision at IoU=0.50 (COCO benchmark)
Inference Time (CPU)	~30ms	Average per 640×640 image on 4-core CPU
Model Size	6.2 MB	Smallest YOLO variant, suitable for cloud CPU deployments
Detection Coverage	≥ 90%	Products detected on demo shelf images
Confidence Threshold	0.25	Low threshold to maximize recall

6.2 CLIP SKU Matching Performance

Metric	Value	Description
Embedding Dimension	512	L2-normalized ViT-B/32 output vector
Similarity Threshold	0.82	Empirically tuned on demo dataset
Inference Time per Crop	~25ms	CLIP forward pass on CPU
Library Search Time	< 5ms	Cosine similarity over all library entries
Stage 2 Categories	38	Zero-shot retail category text prompts
Match Accuracy (demo)	~85%	On demo shelf images with populated library

6.3 Planogram Lookup Performance

Metric	Value	Description
Grid Assignment Method	Proportional	xc/img_w × num_cols (handles YOLO misses)
Demo Planogram Shelves	3	SHELF-BEV-01 (4×8), SHELF-DAIRY-01 (4×8), SHELF-SNACKS-01 (5×8)
Compliance Check	Word-level	With hyphen/underscore normalization
Lookup Latency	< 2ms	Single SQLite query per detected product
Hit Rate (demo)	~70–80%	Products matched via planogram after library miss

6.4 LLM Fallback Performance

Provider	Avg Latency	Accuracy	Cost	Notes
GPT-4o	2–4s	~90%	Paid API	Primary, best visual understanding
Gemini 2.0 Flash	1–3s	~88%	Free tier	Fast, Google ecosystem
Groq LLaMA-3 Vision	< 1s	~80%	Free tier	Fastest, open-weight model
Rule-based fallback	< 1ms	Low	Free	Last resort, CLIP category label

6.5 End-to-End Pipeline Performance

Stage	Latency	Notes
Image Upload + YOLOv8n	~100ms	File I/O + detection inference
CLIP Stage 2 (per crop)	~25ms	Category classification
CLIP Stage 3 (per crop)	~30ms	Embedding + library search
Planogram Lookup (per crop)	< 5ms	SQLite query
LLM Fallback (per unmatched crop)	1–4s	Provider-dependent
Total (20 products, no LLM needed)	~800ms	YOLO + CLIP + Planogram only
Total (20 products, all LLM needed)	~45s	Worst case, all unmatched

7. LLM INTEGRATION AND PROMPTING STRATEGY

7.1 LLM Architecture

The LLM fallback component is invoked only after Stage 3 (CLIP library) and Stage 3.5 (planogram lookup) both fail to identify a product. This gating design minimizes API costs and latency — the vast majority of products on a well-populated shelf are identified by CLIP or planogram before the LLM is needed.

The cascade strategy ensures high uptime: if GPT-4o is rate-limited or unavailable, the system automatically falls through to Gemini 2.0 Flash, then to Groq LLaMA-3. Each provider is given one attempt per product with a 30-second timeout.

7.2 Prompting Strategy

Each LLM receives a structured prompt with the product crop image encoded in base64, requesting JSON output with product_name, brand, category, and confidence fields. The prompt instructs the model to set confidence below 50 if the product label is not clearly readable. Results with confidence < 50 are flagged as 'low confidence' rather than accepted as identifications, preventing hallucinations from polluting results.

```
{
"product_name": "Coca-Cola Classic 330ml",
"brand": "Coca-Cola",
"category": "Soda & Energy Drinks",
"confidence": 92
}
```

7.3 Natural Language Query Interface

The system includes a natural language query (NLQ) interface that allows store managers to ask inventory questions in plain English. The backend translates natural language to SQL using the LLM, with safeguards: only SELECT queries are accepted, a LIMIT 200 clause is injected to prevent runaway queries, and the database schema is included in the prompt as context.

8. END-TO-END WORKFLOW

8.1 New Shelf Scan Workflow

- **Step 1:** User logs in via the web interface using username/password; JWT token is issued
- **Step 2:** User navigates to the Detection page and selects a shelf (e.g., SHELF-BEV-01)
- **Step 3:** User uploads a shelf photo; the system calls /detection/upload, which runs YOLOv8n and returns bounding boxes
- **Step 4:** User clicks 'Run Pipeline'; the system sends bounding boxes to /detection/pipeline
- **Step 5:** For each crop: planogram Stage 3.5 (first) → CLIP Stage 3 (non-planogram items) → LLM Stage 3b (remaining)
- **Step 6:** Results are displayed in a colour-coded table with identification source badges
- **Step 7:** Planogram compliance status (ok / mismatch / no_planogram) is shown per product
- **Step 8:** Statistics panel shows: total detected, library matched, planogram matched, LLM identified, unmatched

8.2 Library Building Workflow

- **Step 1:** User runs a detection on a shelf with a loaded planogram
- **Step 2:** Planogram-matched products show a '■ Save to Library' button; bulk 'Save All' button for all at once
- **Step 3:** System crops the bounding box from the stored event image and computes a CLIP embedding
- **Step 4:** On subsequent pipeline runs, this embedding is included in the cosine similarity search
- **Step 5:** With ≥3 reference images per product, the library achieves 'Good coverage' accuracy

9. FEATURES AND CAPABILITIES

9.1 Core Features

- Four-stage hierarchical detection pipeline with per-product confidence and identification source tracking
- 38-class zero-shot CLIP category classification with brand-specific prompt ensembling
- CLIP image library with cosine similarity SKU matching and progressive library building
- Planogram-first pipeline ordering to prevent cross-shelf false-positive library matches
- Planogram compliance verification with word-level matching and hyphen normalization
- Multi-provider LLM fallback cascade (GPT-4o → Gemini 2.0 Flash → Groq LLaMA-3)
- Name-based category inference override for planogram/library-matched products
- Auto inventory upsert after every pipeline scan

9.2 Advanced Features

- Bulk 'Save All to Library' button for all planogram-matched items in one click
- Natural language query interface (plain English → SQL → inventory results)
- JWT-authenticated multi-user support
- Low-stock alert system with configurable thresholds
- Analytics dashboard with detection event history and identification breakdown
- Planogram CSV import and per-shelf planogram management

9.3 Deployment Features

- Single Docker image for full-stack deployment (FastAPI backend + React SPA)
- Hugging Face Spaces deployment with GitHub Actions CI/CD pipeline
- Lazy-loaded YOLO and CLIP models with in-memory caching
- Environment variable-based API key management for all LLM providers

10. CHALLENGES AND SOLUTIONS

• Cross-shelf CLIP False Positives

Challenge: A CLIP library built from beverage shelf scans would match dairy products at 82–89% cosine similarity, returning wrong beverage names.

Solution: Reordered the pipeline: planogram lookup runs BEFORE CLIP library search. CLIP embeddings are extracted only for products not covered by the planogram, completely eliminating cross-shelf contamination for shelves with a loaded planogram.

• Rank-Based Grid Column Assignment

Challenge: When YOLO missed products in a shelf row, a rank-based column assignment would compress N detected items into columns 1..N, giving wrong (row, col) positions.

Solution: Switched to proportional mapping: col = int(xc / img_width × num_cols). This correctly assigns each product to its absolute column regardless of how many adjacent products YOLO detected.

• Hyphen Normalization in Compliance

Challenge: 'Coca-Cola' (hyphenated in planogram) would fail to match 'Coca Cola' (space-separated as returned by LLM), causing false compliance mismatches.

Solution: Applied replace('-', ' ') and replace('_', ' ') to both detected and expected names before the word-level substring match.

• Incorrect CLIP Stage 2 Categories

Challenge: CLIP's category classification on small, blurry 224×224 bottle crops was unreliable — Fanta Orange was classified as 'Baby Products', Coca-Cola as 'Wine & Spirits'.

Solution: Added _infer_category_from_name(): when a product name is known (from planogram or library), the category is derived via keyword matching rather than the CLIP image guess. Also sharpened CLIP prompts with brand-specific visual descriptors (e.g., 'a red Coca-Cola bottle with white script logo').

• Dependency on GPU for CLIP/YOLO

Challenge: HF Spaces free tier provides only CPU. CLIP inference on CPU was 20–30ms per crop, acceptable but sensitive to concurrency.

Solution: YOLOv8n (6.2MB) and CLIP ViT-B/32 were selected specifically for CPU feasibility. Models are loaded once at startup and kept in memory. Batch processing is used for CLIP embeddings across all crops in a single forward pass.

11. FUTURE SCOPE

11.1 Short-Term Enhancements (3–6 months)

- Fine-tune YOLOv8 on the SKU-110K dataset for higher product bounding box accuracy
- Expand the CLIP library building to include automated web-scraped reference images per product
- Add video stream support for real-time shelf scanning via IP camera feeds
- Implement per-shelf accuracy reporting and trend analysis over time

11.2 Mid-Term Enhancements (6–12 months)

- Train a retail-specific CLIP fine-tuned on product packaging datasets for higher SKU accuracy
- Multi-shelf concurrent scanning with automatic shelf identification from image metadata
- Integration with a POS system for real-time sales-driven inventory depletion tracking
- Mobile application for handheld shelf scanning by store associates

11.3 Long-Term Vision (1–2 years)

- Federated learning across stores to improve the shared product library without sharing raw images
- Automated planogram regeneration based on detected shelf states and historical sales data
- Integration with retail supply chain APIs (SAP, Oracle Retail) for end-to-end inventory management
- GPU-accelerated deployment on HF Spaces Pro for sub-second full-shelf pipeline latency

12. CONCLUSION

12.1 Project Summary

This project successfully demonstrates that a well-orchestrated pipeline of multiple AI models can automate retail shelf intelligence — detecting products, identifying SKUs, verifying planogram compliance, and surfacing actionable inventory insights — without requiring expensive hardware, manual labeling, or per-product classifier training.

12.2 Key Achievements

- Operational four-stage detection pipeline: YOLOv8n → CLIP Stage 2/3 → Planogram → LLM cascade
- Planogram-first ordering that eliminates cross-shelf false positive CLIP library matches
- 38-class zero-shot category classification with brand-specific CLIP prompt ensembling
- Name-based category inference that corrects CLIP's unreliable guesses on small crops
- Progressive library building: planogram-matched products self-populate the CLIP library
- Full-stack web application with JWT auth, analytics, NLQ, and inventory management
- Docker-based deployment with GitHub CI auto-deploy to Hugging Face Spaces

12.3 Learning Outcomes

- Practical experience integrating multiple AI modalities (detection, vision-language, LLMs) in a single production pipeline
- Deep understanding of CLIP's embedding space and the nuances of cosine similarity thresholding
- Real-world experience with retail domain challenges: dense packing, partial occlusion, planogram compliance
- Full-stack development experience: FastAPI, React, SQLAlchemy, Docker, JWT
- MLOps experience: model lazy loading, embedding caching, multi-provider LLM cascades, HF Spaces deployment

12.4 Limitations

- CLIP SKU matching degrades for visually similar product variants (e.g., different flavors of the same brand)
- Planogram grid lookup is approximate — it can mis-assign products near grid boundaries
- LLM hallucination rate of ~5–10% for low-quality or partially occluded product images
- No real-time video processing capability in the current implementation

12.5 Final Thoughts

The key insight of this project is that **orchestration beats individual model performance**. No single model — YOLO, CLIP, or an LLM — solves the retail shelf intelligence problem on its own. The system succeeds because each stage is tuned to its specific subtask, the stages are correctly ordered by cost (fast/cheap stages first, LLM last), and the progressive library building creates a virtuous cycle of improving accuracy. This 'compound AI systems' design pattern is increasingly recognized as the most practical path to reliable AI in production.

13. REFERENCES

13.1 Research Papers

- Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.
- Jocher, G., et al. (2023). YOLOv8: Ultralytics Object Detection Framework. Ultralytics.
- Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML 2021.
- Goldman, E., et al. (2019). Precise Detection in Densely Packed Scenes (SKU-110K). CVPR 2019.
- Ren, S., et al. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015.
- Tan, M., & Le, Q. V. (2020). EfficientDet: Scalable and Efficient Object Detection. CVPR 2020.
- Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- Tonioni, A., & Di Stefano, L. (2019). Domain Invariant Hierarchical Embedding for Retail Product Recognition. CVIU.

13.2 Technical Documentation

- Ultralytics YOLOv8: https://docs.ultralytics.com/
- OpenAI CLIP Repository: https://github.com/openai/CLIP
- FastAPI Documentation: https://fastapi.tiangolo.com/
- React Documentation: https://react.dev/
- Hugging Face Spaces: https://huggingface.co/docs/hub/spaces
- OpenAI GPT-4o API: https://platform.openai.com/docs/
- Google Gemini API: https://ai.google.dev/docs
- Groq API: https://console.groq.com/docs

13.3 Datasets

- SKU-110K: Densely Packed Retail Products — Goldman et al. (2019)
- Planogram demo data: manually curated CSV for 3 demo shelves (beverages, dairy, snacks)

14. APPENDIX

14.1 API Endpoints

Method	Endpoint	Description
POST	/auth/token	Login — returns JWT access token
GET	/auth/me	Get current user info
POST	/detection/upload	Upload shelf image; runs YOLOv8n; returns bounding boxes
POST	/detection/pipeline	Run full 4-stage pipeline on detected crops
POST	/detection/add-to-library	Crop + embed + save to CLIP library
GET	/library/references	List all product library entries
POST	/library/references	Add a new reference image with CLIP embedding
DELETE	/library/references/{id}	Remove a reference image
GET	/library/stats	Library statistics and CLIP model status
GET	/inventory/	List inventory items
GET	/inventory/alerts	Low-stock alerts
POST	/planogram/upload	Import planogram CSV for a shelf
GET	/planogram/{shelf_id}	Get planogram for a shelf
POST	/nlq/query	Natural language → SQL → inventory query
GET	/analytics/dashboard	Aggregate detection statistics
GET	/health	Health check

14.2 Database Schema

```
-- Users
CREATE TABLE users (
id INTEGER PRIMARY KEY, username TEXT UNIQUE,
hashed_password TEXT, created_at TIMESTAMP
);
-- Detection Events
CREATE TABLE detection_events (
id INTEGER PRIMARY KEY, image_path TEXT,
created_at TIMESTAMP, user_id INTEGER REFERENCES users(id)
);
-- Product Library References
CREATE TABLE library_references (
id INTEGER PRIMARY KEY, product_name TEXT, product_id INTEGER,
image_path TEXT, embedding BLOB, created_at TIMESTAMP
);
-- Planogram
CREATE TABLE planogram (
id INTEGER PRIMARY KEY, shelf_id TEXT, row INTEGER, col INTEGER,
product_name TEXT, category TEXT, brand TEXT,
expected_quantity INTEGER, notes TEXT
);
```

14.3 Configuration

```
# .env (local development)
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=AIzaSy...
GROQ_API_KEY=gsk_...
SECRET_KEY=your-jwt-secret-key
DATABASE_URL=sqlite:///./retail_ai.db
CLIP_SIMILARITY_THRESHOLD=0.82
YOLO_CONFIDENCE_THRESHOLD=0.25
YOLO_IOU_THRESHOLD=0.45
LLM_MIN_CONFIDENCE=50
```

14.4 Demo Planogram Format (CSV)

```
shelf_id,row,col,product_name,category,brand,expected_quantity,notes
SHELF-BEV-01,1,1,Coca-Cola Classic,Beverages,Coca-Cola,6,
SHELF-BEV-01,1,2,Pepsi Cola,Beverages,PepsiCo,6,
SHELF-BEV-01,1,3,Sprite Lemon Lime,Beverages,Coca-Cola,6,
SHELF-BEV-01,1,4,Fanta Orange,Beverages,Coca-Cola,4,
SHELF-SNACKS-01,1,1,Lay's Classic Potato Chips,Snacks,Frito-Lay,8,
SHELF-SNACKS-01,1,2,Doritos Nacho Cheese,Snacks,Frito-Lay,8,
```

14.5 Change Log — Recent System Enhancements

Version	Change	Description
v1.0	Initial 4-stage pipeline	YOLOv8n + CLIP Stage 2/3 + LLM fallback cascade
v1.1	Planogram compliance (Stage 3.5)	Grid-based product position lookup against planogram DB
v1.2	Proportional grid mapping	Fixed rank-based column assignment; switched to x-centre / image_width × num_cols
v1.3	Hyphen normalization	replace('-',' ') on both detected and expected names before word-match
v1.4	Auto num_rows / num_cols	Read max(row) and max(col) from planogram DB instead of hardcoded values
v1.5	Save to Library button	Planogram-matched rows show a 'Save to Library' button to bootstrap CLIP library
v1.6	Stage 1 YOLOv8 label fix	Stage 1 card label corrected from 'CLIP Ready' to 'YOLOv8 Ready'
v1.7	Bulk 'Save All to Library'	Single button saves all planogram-matched crops to CLIP library in one click
v1.8	Auto inventory save	_pipeline_inventory_upsert() upserts all identified products to Inventory tables after each scan
v1.9	Planogram-first pipeline	Pipeline reordered: planogram runs before CLIP library. Prevents beverage library matching dairy shelf products.
v2.0	Name-based category inference	_infer_category_from_name() derives category from product name. Brand-specific CLIP prompts sharpened.

14.6 Glossary

Term	Definition
CLIP	Contrastive Language-Image Pre-training — OpenAI model that embeds images and text in a shared vector space
Cosine Similarity	Dot product of two L2-normalized vectors; 1.0 = identical, 0.0 = orthogonal
Planogram	A visual diagram specifying where each product should be placed on a retail shelf
YOLO	You Only Look Once — single-pass object detection architecture
SKU	Stock Keeping Unit — a unique identifier for a specific retail product variant
mAP50	Mean Average Precision at IoU threshold 0.50 — standard object detection metric
LLM	Large Language Model — neural network trained on text (and optionally images) at scale
JWT	JSON Web Token — compact, URL-safe token format for authentication
HF Spaces	Hugging Face Spaces — free platform for hosting ML demo applications
FastAPI	High-performance Python web framework with automatic OpenAPI documentation
ViT	Vision Transformer — attention-based image encoder used in CLIP
IoU	Intersection over Union — measure of bounding box overlap

---

END OF REPORT